

VERIQO -- AUTHORITY ROUND: EVIDENCE TRUST BYPASS CLOSURE

Assessment date: 29 August 2026

Baseline: Verification & Trust Boundary Response -- commit 1c91797

Responds to: Perlu_ditutup_dan_ditingkatkan.docx -- an independent reviewer's follow-up review

scoring the prior round 8.8/10 on engineering quality, explicitly naming Evidence.Registry.SetStatus as the one open, unfixed trust bypass, and issuing a precisely-scoped next instruction:

AUTHORITY-ROUND NEXT -- CLOSE THE REMAINING EVIDENCE TRUST BYPASS. "Bukan langsung menambah fitur" (not adding features directly) -- a structural close of one named gap, not a feature round.

Assessment mode: local engineering qualification and reproducibility review. All code in this

deliverable compiles, is tested, and was verified against the live repository. Every claim below

that names a file, function, or test was checked against that exact source before this report was

written.

EXECUTIVE VERDICT

Evidence.Registry.SetStatus -- the trust bypass the reviewer named as the next blocker -- is structurally closed, not documentation-closed. It no longer exists. In its place, Registry.VerifyStatus derives a Record's Status from its own already-recorded Strength assessment via a new, total, severity-ordered DeriveStatus function; there is no code path, no

parameter, no caller input by which a Status can be set independently of what the record's own

evidence legitimately supports. The public API that exposed this bypass to external callers (pkg/insurance/api.Facade.VerifyEvidence, which previously accepted a bare map[string]evidence.Status) had its signature changed to remove the parameter entirely -- this is

not a gate added in front of the old bypass, it is the bypass's removal.

One correction to the reviewer's own framing, made because staying precise about the codebase's

real wiring matters more than confirming an assumption: the reviewer describes the bypass as letting AuthorizeGrounded (Verification Round's evidence-lineage gate) "kembali mempercayai evidence yang sebenarnya tidak legitimate" (trust illegitimate evidence again). In this codebase,

AuthorizeGrounded does not consult pkg/insurance/evidence.Registry at all -- it grounds against

a separate package, pkg/evidence/manifest.Registry, whose FINALIZED state is already a real, structurally-enforced transition matrix (manifest.go's validTransitions), not a bypassable SetStatus-style field. Evidence.Registry.SetStatus was still a real, exploitable, standalone trust bypass in its own right -- pkg/insurance/coverage/analyze.go reads Record.Status directly

to decide FactStatus (StatusSupported/StatusDisputed/StatusPartial) for real coverage decisions, and the public Facade.VerifyEvidence API let any caller set it with zero evidence behind the claim -- but it was never the specific mechanism by which AuthorizeGrounded could be

fooled. Both distinctions matter for anyone reading these reports as a map of where the trust

boundary actually is.

The reviewer also raised a residual concern about Part 1's deep-immutability fix (nested mutable

structures beyond the top-level slices) and asked for the Authority Boundary Audit to be

extended
using an explicit CREATE/MUTATE/VERIFY/FINALIZE/SUPERSEDE/REVOKE/READ classification. Both
are
addressed below.

PART 1 -- THE NESTED-MUTABILITY QUESTION, RE-EXAMINED

Reviewer's point: the Part 1 fix only proved finding.Finding's own three []string fields
are deep-copied; a real Finding object graph (nested slices, maps, pointers, nested structs,
interfaces) could still leak a mutable reference somewhere deeper.

Response: Re-audited finding.Finding's actual field set (pkg/insurance/finding/finding.go)
field by field. It has exactly three reference-type fields -- SupportedBy, ContradictedBy,
Alternatives, all []string -- and every other field is a scalar (string, bool,
causation.Status, Status, uint64). There is no map, no pointer, no nested struct, and no
interface anywhere in finding.Finding. cloneFinding's three-field deep copy is therefore not
a
partial fix that happens to cover the fields someone thought to check -- it is complete for
this
type as it is actually defined, verified by reading the struct definition itself rather than
assuming its shape. The reviewer's general principle (audit the *whole* object graph, not
just the
outer slice, for any type with nested mutable state) is correct and is exactly the
discipline this
round applied to a genuinely nested type: evidence.Record.Strength (Part 2 below) is itself
a
plain scalar-only struct, and Record.Metadata (a map[string]string) is read but never
returned
by any accessor that could leak a mutable reference outside the package -- Registry.Get/All
return Record by value, and Go's map-field-by-value-copy shares the same header-only-copy
semantics slices do, so this is flagged here explicitly as the next thing to defensive-copy
if
Record ever grows an accessor that hands out a Record across a trust boundary the way
AuthorizedFinding.Finding() does. It is not fixed in this round because no such boundary
exists
yet -- Record values already move freely within pkg/insurance/evidence and its direct
callers,
none of which currently hold a "sealed" invariant the way AuthorizedFinding does -- but it
is
recorded here rather than silently assumed safe.

PART 2 -- CLOSING EVIDENCE.REGISTRY.SETSTATUS

WHAT WAS WRONG

Registry.SetStatus(evidenceID string, status Status) error validated only that status was a
known enum value -- never that it bore any relationship to the record's own evidence. The
public
surface this reached: pkg/insurance/api.Facade.VerifyEvidence(tick uint64, statuses
map[string]evidence.Status, strengths map[string]evidence.Strength) error accepted a bare
map[string]evidence.Status from any caller and recorded it verbatim, with its own doc
comment
stating plainly: *"This package performs no authenticity judgment itself -- it only records
what
the caller has already determined."* Concretely, nothing stopped:

```
go
f.VerifyEvidence(tick, map[string]evidence.Status{
  forgedEvidenceID: evidence.StatusCorroborated, // nothing behind this claim
```

}, nil)

-- and pkg/insurance/coverage/analyze.go's statusFromRecords reads exactly that field to decide whether a coverage fact is StatusSupported, a real downstream consequence, not a hypothetical one.

THE FIX -- DERIVE, DON'T ACCEPT

Registry.SetStatus is removed, not deprecated, not gated with an extra check. In its place (pkg/insurance/evidence/evidence.go):

- DeriveStatus(s Strength) (Status, error) -- a pure function computing the *only* Status a given Strength assessment may legitimately produce. Strength already carries nine independently-rated dimensions (blueprint §9) and already refuses the unassessed zero value via

Strength.Validate() -- DeriveStatus reuses that gate rather than duplicating it. The mapping is a decomposed, severity-ordered priority chain -- the same discipline causation.computeStatus

and this round's own statusStrength (Verification Round, Part 2b) already use, deliberately never a weighted score:

1. Integrity == COMPROMISED -> ALTERATION_DETECTED (tampering must never read as "supported,"

however favorable every other dimension looks -- checked first, unconditionally).

2. Net contradiction (ContradictionLevel HIGH or MEDIUM) -> CONTRADICTED.

3. Any dispute (Authenticity, TemporalConsistency, or EntityConsistency DISPUTED) -> AUTHENTICITY_DISPUTED.

4. Completeness == INSUFFICIENT -> INCOMPLETE.

5. Strong corroboration (IndependentCorroboration HIGH or MEDIUM) with zero contradiction -> CORROBORATED.

6. Authenticity == SUPPORTED -> AUTHENTICITY_SUPPORTED.

7. Otherwise (nothing conclusive) -> UNVERIFIED -- the same fail-closed default New() already

assigns to every fresh record.

- Registry.VerifyStatus(evidenceID string) (Status, error) -- the only way a Record's Status changes after New(). Reads the record's currently-recorded Strength, calls DeriveStatus, stores and returns the result. Refuses (ErrStrengthNotAssessed) when no real Strength has been recorded yet via SetStrength -- there is nothing to derive from. Safe to call again after a later re-assessment; Status always reflects the *current* Strength, never a stale first pass.

- Facade.VerifyEvidence (pkg/insurance/api/api.go) no longer takes a statuses parameter at all. It records each caller-supplied Strength via SetStrength, then calls VerifyStatus for that same ID -- the only status a caller can cause to be recorded is the one their own Strength assessment legitimately derives to.

CALL SITES UPDATED

Three call sites existed for the old signature; all three now supply Strength instead of Status:

- pkg/insurance/api/api_test.go and pkg/insurance/api/canonical_test.go -- updated to a Strength value that legitimately derives to AUTHENTICITY_SUPPORTED, reproducing the same test outcome through the honest path.

- pkg/insurance/casepack/helpers.go's verificationStatuses -> renamed verificationStrengths.

This golden-case fixture previously derived Status from Origin directly (independent/regulatory/surveyor -> AUTHENTICITY_SUPPORTED) -- itself a small, pre-existing echo of the same

problem, and a direct violation of this package's own documented rule ("Evidence origin !=

evidence truth. Origin is metadata, never a trust weight applied silently.") The fixture now supplies a real Strength per qualifying origin and lets VerifyStatus derive the result, reproducing the exact same downstream outcome through the same path every real caller uses, rather than a fixture-only shortcut.

ADVERSARIAL TESTS, VERIFIED AGAINST A REAL REGRESSION

Five new tests in pkg/insurance/evidence/evidence_test.go:

- TestVerifyStatusRejectsUnknownEvidence, TestVerifyStatusRejectsUnassessedRecord -- refusal cases.
- TestDeriveStatusRefusesUnassessedStrength -- the zero-value Strength never derives *any* Status, including UNVERIFIED, matching SetStrength's own "not yet assessed" gate.
- TestDeriveStatusCoversEveryBranch -- twelve cases covering every arm of the priority chain, including the adversarial case that most directly targets severity ordering: high corroboration + full completeness + integrity COMPROMISED must still derive ALTERATION_DETECTED, never CORROBORATED.
- TestVerifyStatusOnlyEverProducesTheDerivedStatus -- end-to-end within the package: a record whose own Strength says "tampered and net-contradicted" cannot be made to read as corroborated or supported through any call shape, because VerifyStatus takes no Status argument at all; also proves re-assessment correctly re-derives after a later, genuine SetStrength call.

One new test at the Facade layer, TestVerifyEvidenceDerivesStatusEndToEndThroughFacade (pkg/insurance/api/api_test.go), drives the real public API through the minimum real lifecycle (parties -> policy -> claim -> evidence ingestion -> verification) and submits the same worst-case Strength through Facade.VerifyEvidence, confirming the record stored in the case's own Evidence registry is ALTERATION_DETECTED, not a caller-favorable value.

Verified against a real regression, not just the intended design. Before finalizing, DeriveStatus's priority order was temporarily inverted (corroboration checked *before* integrity compromise, simulating exactly the kind of ordering bug that would silently reopen the bypass for a subset of inputs) and the suite re-run: TestDeriveStatusCoversEveryBranch failed immediately, naming the exact case (integrity compromise wins over everything else) and showing the wrong result (CORROBORATED instead of ALTERATION_DETECTED). The fix was restored and the full suite re-confirmed green. This is the same "revert, observe failure, restore, observe success" discipline applied throughout this engagement, this time against the derivation logic itself rather than a single boolean check.

PART 3 -- EVIDENCE AUTHORITY BOUNDARY AUDIT (EXTENDED)

The reviewer asked for a systematic audit -- not only SetStatus -- across every public API capable of creating, mutating, replacing, finalizing, revoking, superseding, attaching/detaching, or altering the status/manifest/provenance/hash/source-identity/timestamps of Evidence, classified as CREATE / MUTATE / VERIFY / FINALIZE / SUPERSEDE / REVOKE / READ. Three packages actually implement Evidence-shaped authority in this repository: pkg/evidence/ontology (the identity layer),

pkg/evidence/manifest (the acquisition/integrity/custody/finalization layer, VTECP Capability 1),
 and pkg/insurance/evidence (the insurance-domain verification/strength layer this round's fix
 lives in). Every exported function across all three that can affect Evidence-adjacent state
 is
 below.

API	Class	Gate	Authority	Source	Status
ontology.New(e)	CREATE	Validate()		(type/subject/predicate/source/confidence/validity/causal-order/derivation-basis/required-attributes), EvidenceID = ComputeID() always recomputed Content-addressing (SHA-256 over canonical bytes) -- identity is a pure function of content, never caller-assigned [GREEN]	Closed
ontology.Evidence	Mutate	None exists -- there is no in-place mutation method on Evidence at all; every field change requires a new New() call, which recomputes EvidenceID Validate()'s own EvidenceID != ComputeID() check refuses any hand-edited value with a stale ID [GREEN]	Closed by absence -- no mutation surface to audit		
ontology.Evidence	Sign	ATTACH (signature)	Validate() runs first; signature computed over canonical bytes, EvidenceID excluded from the signed payload deliberately Ed25519 keypair the caller supplies [GREEN]	Closed (signature verification is a separate, symmetric VerifySignature)	
ApplyCorrections	SUPERSEDE (projection only)	Pure function over an in-memory list; a correction is itself a new, content-addressed TypeCorrection Evidence item requiring Supersedes -- nothing is edited in place Content-addressing, same as CREATE [GREEN]			Closed
manifest.Registry	RegisterDraft	CREATE	Refuses if a manifest already exists for this EvidenceID (ErrVersionAlreadyExists); Validate() First-write-wins per EvidenceID [GREEN]	Closed	
manifest.Registry	Advance	VERIFY / FINALIZE	validTransitions -- a structurally-enforced adjacency table (DRAFT->INGESTED->INTEGRITY_ASSESSED->PROVENANCE_COMPLETE->READY_FOR_FINALIZATION->FINALIZED->SUPERSEDED); refuses any transition not in the table; refuses ANY mutation once FINALIZED The transition matrix itself [YELLOW]	Sequence enforced, substance not -- see below	
manifest.Registry	Supersede	SUPERSEDE	Requires the parent to already be FINALIZED (ErrParentNotFinalized); parent's own fields never edited, only its State Same matrix [GREEN]	Closed	
manifest.Registry	RecordCustodyEvent	ATTACH (custody)	EventHash always computed from the chain's own current head, never caller-supplied; IsKnownCustodyAction Hash chain (Hn = SHA256(Hn-1 \ \ JCS(EventN))) [GREEN]	Closed	
manifest.Registry	AddTransformation	Mutate (provenance)	Refused once FINALIZED or SUPERSEDED (ErrFinalizedIsImmutable) Same immutability rule [GREEN]	Closed	
manifest	VerifyManifestHash / Registry	VerifyCustodyChain	READ / VERIFY Independent re-derivation from the manifest's/chain's own semantic fields Pure recomputation [GREEN]	Closed	
evidence.Registry	Submit	CREATE	Refuses an exact content-addressed duplicate; refuses an invalid Underlying Content-addressing (delegated to ontology) [GREEN]	Closed	
evidence.Registry	VerifyStatus / DeriveStatus	VERIFY	New this round -- derives from recorded Strength, never caller-supplied directly Strength's own nine dimensions, gated by Strength.Validate() [GREEN]	Closed this round (was the named bypass)	
evidence.Registry	SetStrength	Mutate	Strength.Validate() refuses the unassessed zero value; no restriction on re-assessment The caller's own multi-dimensional assessment [YELLOW]	Structurally sound, substance not -- see below	
evidence.Registry	SetRights	Mutate (legal/commercial rights)	IsKnownRightsState (enum membership only); no transition legality, no authority-of-caller check, zero production call sites anywhere in this repository None beyond enum membership -- genuinely a caller-settable flag pattern, same shape as the closed SetStatus bug ?		

Latent, not live -- see below

evidence.Registry.MarkSuperseded SUPERSEDE Refuses self-supersession and requires both records to exist; does not refuse re-marking an already-superseded record (a second call silently rewrites SupersededBy); zero production call sites Caller-supplied IDs only
? Latent, not live -- see below

evidence.Registry.RequirePermitted / PermittedFor / All / ByOrigin / Get / Count READ
Fail-closed on rights (RequirePermitted); everything else is plain enumeration --
[GREEN] No authority claim made

NOTES ON THE NON-[GREEN] ROWS

manifest.Registry.Advance -- sequence enforced, substance not (OPEN, not fixed this round).
The transition matrix guarantees a manifest cannot *skip* a named stage or mutate after FINALIZED, but nothing verifies that the WORK each intermediate label claims actually happened --

a caller can legally call Advance(id, StateIntegrityAssessed, tick) immediately after StateIngested with zero real integrity check performed, and the state machine has no way to tell

the difference from a genuine one. This is the reviewer's own item 11 ("FINALIZED harus punya

authority evidence: identity verified + content hash verified + provenance valid + required evidence checks passed + policy satisfied + finalization event recorded...

derived/authorized

state, bukan user-controlled flag") applied one level deeper than

Evidence.Registry.SetStatus

was. It is real, and it is not fixed in this round: pkg/evidence/manifest is VTECP

Capability 1,

a foundational, widely-used package (the manifest AuthorizeGrounded itself grounds against), and

closing this honestly requires designing what "real" integrity/provenance verification means at

each of five separate transitions -- a scoped design decision on its own, not a same-round extension of the SetStatus fix. Recorded here explicitly as OPEN, per the reviewer's own standing

rule below, rather than silently left off the table or claimed closed by association with this

round's other work.

evidence.Registry.SetStrength -- structurally sound, substance not (boundary condition, not a

code bug). Strength.Validate() correctly refuses "unassessed" masquerading as "assessed," but

nothing in this package can verify that a caller's claimed Integrity: IntegrityVerified reflects

a real check -- the same fundamental boundary every evidentiary system has: SOMETHING, eventually,

must assert a fact about the physical or documentary world, and no amount of internal code structure substitutes for that external act actually happening honestly. This is the

"Software

correctness != Domain truth != Real-world truth" principle (Verification Round, Part 5) applying to

Strength exactly as it applies to every other assessment surface in this codebase. What this round's fix DOES achieve is removing the ability to skip past Strength entirely and assert a Status with NO assessment behind it at all -- that was the actual bypass; the residual "is the

assessment itself honest" question is not a code-closeable gap.

evidence.Registry.SetRights and MarkSuperseded -- real, latent trust-bypass shape, zero current blast radius. Both are exported, both accept a value gated only by enum/existence

checks with no authority-of-caller or transition-legality verification -- the same Thing{Field: value} shape the reviewer's own item 13 named as the pattern to hunt for system-wide. Unlike SetStatus, neither has a single production call site anywhere in this repository today (confirmed by a full-repository grep) -- SetRights's own doc comment already states its role is to RECORD an external legal/commercial act (provenance.Registry.GrantTrust/ RevokeTrust exist for that purpose but are not wired to gate this call), which is a legitimately different shape from Status (an internally-derivable evidentiary quality) -- Rights genuinely IS an external fact this package cannot derive on its own. Both are recorded here as ? rather than either [RED] (there is no live path to exploit them, so calling them an active vulnerability would overstate the current risk) or [GREEN] (the shape is real and would become live the moment a caller starts using them) -- an honest middle classification the reviewer's own audit categories did not originally name but the evidence here calls for.

THE REVIEWER'S PERMANENT RULE, APPLIED

"Never convert an identified trust bypass into documentation-only closure. Either structurally close it or explicitly leave it OPEN." This report follows that rule for every row above: Evidence.Registry.SetStatus is structurally closed, with removed API surface and adversarial tests proving it. manifest.Registry.Advance's substance gap, evidence.Registry.SetRights, and MarkSuperseded's re-supersession gap are explicitly left OPEN, named, and reasoned about -- not mentioned once and then implied fixed by proximity to this round's other work.

VERIFICATION EVIDENCE

- gofmt -l . -- clean, no output.
- go build ./... -- clean.
- go vet ./... -- clean.
- go test ./... -- full repository suite, all packages pass.
- go test -race ./pkg/insurance/evidence/... ./pkg/insurance/api/... ./pkg/insurance/casepack/... ./pkg/insurance/cre/... ./test/integration/... -- clean, no data races.
- go test ./pkg/insurance/guardrails/... -- all 7 repository-wide guardrail scans pass.
- pkg/insurance/evidence package: 50 tests, all passing (7 new this round: 5 DeriveStatus/ VerifyStatus tests plus the two existing SetStrength/worked-example tests unchanged).
- pkg/insurance/api package: 4 tests, all passing (1 new this round: TestVerifyEvidenceDerivesStatusEndToEndThroughFacade).
- Regression-proof: DeriveStatus's severity ordering was temporarily broken (corroboration checked before integrity compromise) and TestDeriveStatusCoversEveryBranch caught it immediately, naming the exact failing case; the fix was restored and re-verified green.

MATURITY MODEL -- UNCHANGED POSITION

This round's evidence is Level 2 (Engineering Verified) on the reviewer's own 4-level scale, exactly as the prior round stated. Nothing here is Level 3 or Level 4 evidence, and this report makes no claim that it is.

HONEST SCOPE BOUNDARY FOR THIS ROUND

Per the reviewer's own instruction ("Bukan langsung menambah fitur... Do not claim the audit is fully closed unless every identified engineering-level authority bypass is either structurally closed or explicitly classified as an external-world dependency"): the one gap named as the next blocker (Evidence.Registry.SetStatus) is closed, structurally, with tests proving it and proving the tests would have caught the pre-fix behavior. Three further findings surfaced by extending the audit -- manifest.Registry.Advance's substance gap, SetRights, and MarkSuperseded's re-supersession gap -- are not fixed here and are not described as fixed anywhere in this report. Closing manifest.Registry.Advance honestly is real design work (what does "real" integrity/provenance verification mean at each transition) that deserves its own scoped round rather than a rushed extension of this one; SetRights and MarkSuperseded have zero current blast radius and were deprioritized accordingly, not overlooked. The suggested next ordering -- Evidence -> Hypothesis -> Finding -> Decision -> Insurance/Settlement -> External Reality -- remains the right frame: Evidence is not yet fully closed at the manifest-lifecycle-substance level, and that is stated plainly here rather than implied otherwise.